St. XAVIER'S
CATHOLIC COLLEGE OF ENGINEERING
(Autonomous)
Chunkankadai, Nagercoil - 629 003
Kanyakumari District, Tamil Nadu

Approved by AICTE & Affiliated to Anna University, Chennai
Accredited with 'A' Grade by NAAC
UG Programs (ECE, EEE, Mech, Civil, CSE & IT) Accredited by NBA
Anna University Recognized Research Institute
Recognized under section 2(f) & 12(B) of UGC Act, 1956,
UG Programs (ECE, EEE, Mech, Civil, CSE & IT),
MBA & MCA Programs Permanently Affiliated

DEPARTMENT OF .....

LABORATORY RECORD NOTEBOOK

20 - 20

Certified that this is a bonafide record of the practical work done by

Mr. / Ms....., Register Number .....

Studying ..... Semester ..... Degree programme in the

Department of ..... for the

Subject .....

Staff-in-charge **Head of the Department**

Submitted for the Semester Examination held on .....

Internal Examiner **External Examiner**

IT22601 – DATA SCIENCE

L	T	P	C
2	0	2	3

COURSE OUTCOMES:

At the end of the course, the students will be able to:

CO1: Describe the data science process.

CO2: Explore different types of data description for data science process.

CO3: Build knowledge on relationships between data.

CO4: Use the Python Libraries for Data Wrangling.

CO5: Apply visualization Libraries in Python to interpret and explore data. CO6 Infer different visualization tools and techniques to leverage the visualization of data set.

COURSE OBJECTIVES:

- To understand the data science fundamentals and process
- To learn to describe the data for the data science process
- To learn to describe the relationship between data
- To utilize the Python libraries for Data Wrangling
- To present and interpret data using visualization libraries in Python

CONTENTS

Expt. No.	Date of Experiment	Name of the Experiment	CO Mapping	Page No.	Marks Awarded	Course Incharge Sign. with Date

EXNO:1 **DATE:**

WORKING WITH NUMPY ARRAYS

AIM

To Learn fundamentals and write programs using Numpy arrays

1. Find the maximum and minimum element from an array using numpy

Algorithm:

1. Import NumPy library.
2. Create a NumPy array.
3. Assume first element as maximum and minimum.
4. Traverse the array.
5. Compare each element with max and min.
6. Update values accordingly.
7. Display maximum and minimum elements.

Program:

```
import numpy as np
arr = np.array([10, 25, 5, 40, 15])
maximum = arr[0]
minimum = arr[0]
for i in range(len(arr)):
    if arr[i] > maximum:
        maximum = arr[i]
    if arr[i] < minimum:
        minimum = arr[i]
print("Maximum element:", maximum)
print("Minimum element:", minimum)
```

2. Implement linear search wihtout using builtin functions

Algorithm:

1. Import NumPy library.
2. Create a NumPy array.
3. Read the element to be searched.
4. Traverse the array element by element.
5. If element matches, print its position.
6. If not found till the end, print “Not Found”.

Program:

```
import numpy as np
```

```
arr = np.array([10, 20, 30, 40, 50])
key = 30
found = False
for i in range(len(arr)):
    if arr[i] == key:
        print("Element found at index:", i)
        found = True
        break
if not found:
    print("Element not found")
```

3. Implement sorting an array without using builtin functions

Algorithm:

1. Import NumPy.
2. Create a NumPy array.
3. Compare adjacent elements.
4. Swap if the first is greater than the second.
5. Repeat until the array is sorted.
6. Display sorted array.

Program:

```
import numpy as np
arr = np.array([5, 2, 9, 1, 6])
 n = len(arr)
for i in range(n):
    for j in range(0, n - i - 1):
        if arr[j] > arr[j + 1]:
            temp = arr[j]
            arr[j] = arr[j + 1]
            arr[j + 1] = temp
print("Sorted array:", arr)
```

4. Implement matrix operations such as addition,subtraction,multiplication, and transpose

Algorithm

1. Import NumPy library.
2. Create two matrices using NumPy arrays.
3. Perform matrix addition.
4. Perform matrix subtraction.
5. Perform matrix multiplication.
6. Find transpose of both matrices.
7. Display all results.

Program:

```
import numpy as np
A = np.array([[1, 2],
      [3, 4]])
B = np.array([[5, 6],
      [7, 8]])
add = A + B
sub = A - B
mul = A @ B
transA = A.T
transB = B.T
print("Matrix A:\n", A)
print("Matrix B:\n", B)
print("\nAddition of A and B:\n", add)
print("\nSubtraction of A and B:\n", sub)
print("\nMultiplication of A and B:\n", mul)
print("\nTranspose of A:\n", transA)
print("\nTranspose of B:\n", transB)
```

5. Find the sum and average of array element

Algorithm:

1. Start
2. Import the NumPy library
3. Create a NumPy array with elements
4. Initialize a variable `total` to 0
5. Find the length of the array
6. Traverse the array using a loop
7. Add each element to `total`
8. Calculate average using:
   average = total / number of elements
9. Display the sum
10. Display the average
11. Stop

Program:

```
import numpy as np
    arr = np.array([10, 20, 30, 40, 50])
     total = 0
for i in range(len(arr)):
   total += arr[i]
average = total / len(arr)
print("Sum:", total)
print("Average:", average)
```

output:

1.Maximum element: 40
Minimum element: 5

```
2. Element found at index: 2
3. Sorted array: [1 2 5 6 9]
4. Matrix A:
 [[1 2]
  [3 4]]
       Matrix B:
 [[5 6]
  [7 8]]
      Addition of A and B:
 [[ 6  8]
  [10 12]]
      Subtraction of A and B:
 [[-4 -4]
  [-4 -4]]
Multiplication of A and B:
 [[19 22]
  [43 50]]
Transpose of A:
 [[1 3]
  [2 4]]
    Transpose of B:
 [[5 7]
  [6 8]]
5. Sum: 150
   Average: 30.0
```

CONCLUSION:

Thus to learn fundamentals and write program using numpy arrays has been executed and verified successfully.

EX NO:2 **DATE:**

WORKING WITH PANDAS DATA FRAMES

AIM

To work with all the functions and methods that are likely to be used in the data analysis process by using the customer churn dataset.

1. Create a simple Pandas DataFrame

```
import pandas as pd
data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}
#load data into a DataFrame object:
df = pd.DataFrame(data)
print(df)
```

2. Locate one Row

```
print(df.loc[0])
```

3. **Locate multiple Rows**

```
print(df.loc[[0, 1]])
```

4. Load CSV Files Into a DataFrame

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df)
```

5. print the entire DataFrame using to_string()

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.to_string())
```

6. Find max_rows

```
import pandas as pd
print(pd.options.display.max_rows)
```

7. printing the first 10 rows of the DataFrame:

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.head(10))
```

8. . Print the last 5 rows of the DataFrame:

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.tail())
```

9. Find more information about the data set.

```
print(df.info())
```

10. remove rows that contain empty cells

```
import pandas as pd
df = pd.read_csv('data.csv')
new_df = df.dropna()
print(new_df.to_string())
```

11. Replace empty cells with a value

```
import pandas as pd
df = pd.read_csv('data.csv')
df.fillna(130, inplace = True)
```

12. Replace Only For Specified Columns

```
import pandas as pd
df = pd.read_csv('data.csv')
df.fillna({"Calories": 130}, inplace=True)
```

13. Calculate the MEAN, and replace any empty values

```
import pandas as pd
df = pd.read_csv('data.csv')
x = df["Calories"].mean()
df.fillna({"Calories": x}, inplace=True)
print(df.to_string())
```

14. **Calculate the MEDIAN, and replace any empty values**

```
import pandas as pd
df = pd.read_csv('data.csv')
x = df["Calories"].median()
df.fillna({"Calories": x}, inplace=True)
```

15. Remove duplicates

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.duplicated())
```

16. **Grouping with groupby()**

```
import pandas as pd
data = {
    "Department": ["CSE", "ECE", "CSE", "EEE", "ECE", "CSE"],
    "Student": ["A", "B", "C", "D", "E", "F"],
    "Marks": [85, 78, 90, 70, 88, 95]
}
df = pd.DataFrame(data)
grouped = df.groupby("Department")
print(grouped)
```

Output :

Untitled0.ipynb_

```
   calories  duration
0       420        50
```

```
1      380       40
2      390       45
```

```
calories    420
duration     50
Name: 0, dtype: int64
```

```
    Duration  Pulse  Maxpulse  Calories
0         60    110       130     409.1
1         60    117       145     479.0
2         60    103       135     340.0
3         45    109       175     282.4
4         45    117       148     406.0
..       ...    ...       ...       ...
164       60    105       140     290.8
165       60    110       145     300.0
166       60    115       145     310.2
167       75    120       150     320.4
168       75    125       150     330.4

[169 rows x 4 columns]
```

```
    Duration  Pulse  Maxpulse  Calories
0         60    110       130     409.1
1         60    117       145     479.0
2         60    103       135     340.0
3         45    109       175     282.4
4         45    117       148     406.0
5         60    102       127     300.0
6         60    110       136     374.0
7         45    104       134     253.3
8         30    109       133     195.1
9         60     98       124     269.0
10        60    103       147     329.3
```

```
    Duration  Pulse  Maxpulse  Calories
164       60    105       140     290.8
165       60    110       145     300.0
166       60    115       145     310.2
167       75    120       150     320.4
168       75    125       150     330.4
```

```
0   Duration  169 non-null    int64
 1   Pulse     169 non-null    int64
```

```
 2   Maxpulse  169 non-null    int64
 3   Calories  164 non-null    float64
dtypes: float64(1), int64(3)
memory usage: 5.4 KB
None
```

```
   Duration  Pulse  Maxpulse  Calories
0        60    110       130     409.1
1        60    117       145     479.0
2        60    103       135     340.0
3        45    109       175     282.4
4        45    117       148     406.0
```

```
     Duration  Pulse  Maxpulse     Calories
0         60    110       130   409.100000
1         60    117       145   479.000000
2         60    103       135   340.000000
3         45    109       175   282.400000
4         45    117       148   406.000000
5         60    102       127   300.000000
6         60    110       136   374.000000
7         45    104       134   253.300000
8         30    109       133   195.100000
9         60     98       124   269.000000
10         60    103       147   329.300000
```

```
   Duration  Pulse  Maxpulse  Calories
0        60    110       130     409.1
1        60    117       145     479.0
2        60    103       135     340.0
3        45    109       175     282.4
4        45    117       148     406.0
```

```
0      False
1      False
2      False
3      False
4      False
       ...
164    False
165    False
166    False
167    False
168    False
```

```
Length: 169, dtype: bool
```

```
Aggregated mean marks per department:
Department
CSE    90.0
ECE    83.0
EEE    70.0
Name: Marks, dtype: float64
```

```
   Duration  Pulse  Maxpulse  Calories
0        60    110       130     409.1
1        60    117       145     479.0
2        60    103       135     340.0
3        45    109       175     282.4
4        45    117       148     406.0
5        60    102       127     300.0
6        60    110       136     374.0
7        45    104       134     253.3
8        30    109       133     195.1
9        60     98       124     269.0
```

```
0      False
1      False
2      False
3      False
4      False
      ...
164    False
165    False
166    False
167    False
168    False
Length: 169, dtype: bool
```

```
Aggregated mean marks per department:
Department
CSE    90.0
ECE    83.0
EEE    70.0
Name: Marks, dtype: float64
```

Exercise:

1. IMPLEMENT STUDENT RESULT ANALYSIS

1. Calculate Total Marks
2. Calculate Average Marks
3. Assign Grades
4. Rank Students based on Total Marks
5. Sort by Rank
6. Group by Department and find average marks

Code:

```
import pandas as pd
# 1. Create Student Dataset
df = pd.read_csv('student_data.csv')
print("\n1. Student DataFrame")
print(df)
# 2. Calculate Total Marks
df["Total"] = df["Maths"] + df["Science"] + df["English"]

print("\n2. Total Marks")
print(df)

# 3. Calculate Average Marks
df["Average"] = df["Total"] / 3

print("\n3. Average Marks")
print(df)

# 4. Assign Grades
 def grade(avg):
     if avg >= 90:
        return "A"
    elif avg >= 75:
        return "B"
    elif avg >= 60:
        return "C"
    else:
        return "D"

df["Grade"] = df["Average"].apply(grade)

print("\n4. Grades Assigned")
print(df)

# 5. Rank Students based on Total Marks
```

```
df["Rank"] = df["Total"].rank(ascending=False)

print("\n5. Student Ranking")
print(df)
```

 # 6. Sort Students by Rank

```
df_sorted = df.sort_values(by="Rank")

print("\n6. Students Sorted by Rank")
print(df_sorted)
```

 # 7. Group by Department and find average marks

```
print("\n7. Average Marks Department-wise (groupby)")
dept_avg = df.groupby("Department")["Average"].mean()
print(dept_avg)
```

Output :

```
1. Student DataFrame
   RollNo       Name Department  Maths  Science  English
0     101       Todd        EEE     73       90       58
1     102       Mary        ECE     76       69       87
2     103    Patrick        CSE     79       57       59
3     104     Gloria       MECH     55       87       50
4     105    Leonard        ECE     64       54       87
..    ...        ...        ...    ...      ...      ...
95    196     Joseph        ECE     70       58       85
96    197     Ashley        EEE     57       82       82
97    198    Michael        CSE     59       52       88
98    199    William        EEE     75       63       53
99    200  Stephanie        CSE     55       58       63

[100 rows x 6 columns]

2. Total Marks
   RollNo       Name Department  Maths  Science  English  Total
0     101       Todd        EEE     73       90       58    221
1     102       Mary        ECE     76       69       87    232
2     103    Patrick        CSE     79       57       59    195
3     104     Gloria       MECH     55       87       50    192
4     105    Leonard        ECE     64       54       87    205
..    ...        ...        ...    ...      ...      ...    ...
95    196     Joseph        ECE     70       58       85    213
96    197     Ashley        EEE     57       82       82    221
```

97	198	Michael	CSE	59	52	88	199
98	199	William	EEE	75	63	53	191
99	200	Stephanie	CSE	55	58	63	176

[100 rows x 7 columns]

3. Average Marks

	RollNo	Name	Department	Maths	Science	English	Total	Average
0	101	Todd	EEE	73	90	58	221	73.666667
1	102	Mary	ECE	76	69	87	232	77.333333
2	103	Patrick	CSE	79	57	59	195	65.000000
3	104	Gloria	MECH	55	87	50	192	64.000000
4	105	Leonard	ECE	64	54	87	205	68.333333
..	...	...	...	...	...	...	...	...
95	196	Joseph	ECE	70	58	85	213	71.000000
96	197	Ashley	EEE	57	82	82	221	73.666667
97	198	Michael	CSE	59	52	88	199	66.333333
98	199	William	EEE	75	63	53	191	63.666667
99	200	Stephanie	CSE	55	58	63	176	58.666667

[100 rows x 8 columns]

4. Grades Assigned

	RollNo	Name	Department	Maths	Science	English	Total	Average \
0	101	Todd	EEE	73	90	58	221	73.666667
1	102	Mary	ECE	76	69	87	232	77.333333
2	103	Patrick	CSE	79	57	59	195	65.000000
3	104	Gloria	MECH	55	87	50	192	64.000000
4	105	Leonard	ECE	64	54	87	205	68.333333
..	...	...	...	...	...	...	...	...
95	196	Joseph	ECE	70	58	85	213	71.000000
96	197	Ashley	EEE	57	82	82	221	73.666667
97	198	Michael	CSE	59	52	88	199	66.333333
98	199	William	EEE	75	63	53	191	63.666667
99	200	Stephanie	CSE	55	58	63	176	58.666667

	Grade
0	C
1	B
2	C
3	C
4	C
..	...
95	C

```
96     C
97     C
98     C
99     D

[100 rows x 9 columns]
```

5. Student Ranking

```
   RollNo       Name Department  Maths  Science  English  Total     Average  \
0     101       Todd        EEE     73       90       58    221  73.666667
1     102       Mary        ECE     76       69       87    232  77.333333
2     103    Patrick        CSE     79       57       59    195  65.000000
3     104     Gloria       MECH     55       87       50    192  64.000000
4     105    Leonard        ECE     64       54       87    205  68.333333
..    ...        ...        ...    ...      ...      ...    ...        ...
95    196     Joseph        ECE     70       58       85    213  71.000000
96    197     Ashley        EEE     57       82       82    221  73.666667
97    198    Michael        CSE     59       52       88    199  66.333333
98    199    William        EEE     75       63       53    191  63.666667
99    200  Stephanie        CSE     55       58       63    176  58.666667

   Grade  Rank
0      C  55.0
1      B  32.5
2      C  87.0
3      C  89.5
4      C  78.0
..   ...   ...
95     C  63.5
96     C  55.0
97     C  83.0
98     C  91.0
99     D  98.0

[100 rows x 10 columns]
```

6. Students Sorted by Rank

```
   RollNo       Name Department  Maths  Science  English  Total     Average  \
32    133      Dylan        CSE    100      100       95    295  98.333333
63    164       Seth        EEE     97       84       97    278  92.666667
88    189     Robert        EEE     87       84       99    270  90.000000
86    187     Ronald        CSE     87       81       98    266  88.666667
59    160    William        CSE     97       76       91    264  88.000000
..    ...        ...        ...    ...      ...      ...    ...        ...
```

```
48    149    Sharon      EEE    72     52     59    183  61.000000
23    124    Olivia      ECE    55     67     60    182  60.666667
99    200  Stephanie     CSE    55     58     63    176  58.666667
61    162     Lynn      CIVIL   61     51     62    174  58.000000
46    147    Jason      CIVIL   59     65     50    174  58.000000

   Grade  Rank
32     A   1.0
63     A   2.0
88     A   3.0
86     B   4.0
59     B   5.0
..  ...   ...
48     C  96.0
23     C  97.0
99     D  98.0
61     D  99.5
46     D  99.5

[100 rows x 10 columns]

7. Average Marks Department-wise (groupby)
Department
CIVIL    72.733333
CSE      77.216667
ECE      72.916667
EEE      73.619048
MECH     73.680000
Name: Average, dtype: float64
```

2. **IMPLEMENT EMPLOYEE SALARY ANALYSIS**

1. Calculate Total Salary
2. Calculate Average Salary
3. Assign Salary Grade
4. Rank Employees by Total Salary
5. Sort Employees by Rank
6. Average Total Salary Department-wise

Code :

```
import pandas as pd

df = pd.read_csv('employee_dataset.csv')
```

```
df['TotalSalary'] = df['BasicSalary'] + df['Bonus']

average_salary = df['TotalSalary'].mean()
print("Average Total Salary:", average_salary)

def assign_grade(salary):
    if salary >= 8000:
        return 'A'
    elif salary >= 6000:
        return 'B'
    else:
        return 'C'

df['SalaryGrade'] = df['TotalSalary'].apply(assign_grade)

df['Rank'] = df['TotalSalary'].rank(ascending=False, method='dense')

df_sorted = df.sort_values(by='Rank')

dept_avg = df.groupby('Department')['TotalSalary'].mean().reset_index()
dept_avg.rename(columns={'TotalSalary': 'DeptAverageSalary'}, inplace=True)

final_df = df_sorted.merge(dept_avg, on = 'Department',how = 'left')
final_df
```

Output :

```
Average Total Salary: 77507.57
```

	EmpID	Name	Department	BasicSalary	Bonus	TotalSalary	SalaryGrade	Rank	DeptAverageSalary
0	159	Christine	Finance	98135	13150	111285	A	1.0	76776.333333
1	116	Debbie	HR	94896	11519	106415	A	2.0	79067.705882
2	182	Michael	Finance	98361	7482	105843	A	3.0	76776.333333
3	148	Jason	Admin	91543	13763	105306	A	4.0	80153.285714
4	199	Michele	Admin	99043	5942	104985	A	5.0	80153.285714
...	...	...	...	...	...	...	...	...	...
95	136	Tara	Admin	46389	3618	50007	A	96.0	80153.285714
96	173	Brian	HR	44555	4442	48997	A	97.0	79067.705882
97	151	Stuart	Marketing	43116	5534	48650	A	98.0	71697.954545
98	186	Tammy	Admin	41510	6636	48146	A	99.0	80153.285714
99	133	Lawrence	Marketing	43028	3445	46473	A	100.0	71697.954545

100 rows × 9 columns

CONCLUSION

Thus, the various functions using Pandas data frames were executed and the outputs were verified successfully.

EX NO:3 **DATE:**

DESCRIPTIVE ANALYTICS ON THE IRIS DATA SET

AIM

To read the Iris data set from text files, Excel, and web sources and perform descriptive analytics using various Python commands.

The **Iris dataset** contains 150 observations of iris flowers with four attributes:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

  and one categorical variable:
- Species (Setosa, Versicolor, Virginica)

Exercise:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

1. Reading data from text files

```
iris_txt = pd.read_csv("/content/iris.data.txt", delimiter=",")
iris_txt.head()
```

2. Reading data from Excel

```
iris_excel = pd.read_excel("Iris.xlsx")
iris_excel.head()
```

3. Reading data from the web

```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
iris_web = pd.read_csv(url)
iris_web.head()
```

4. Descriptive Statistics

```
iris_web.describe()
```

5. Measures of Central Tendency

```
iris_web.mean(numeric_only=True)
iris_web.median(numeric_only=True)
iris_web.mode()
```

6. Measures of Dispersion

```
iris_web.var(numeric_only=True)
iris_web.std(numeric_only=True)
```

7. Group-wise Analysis by Species

```
iris_web.groupby("species").mean()
```

8. Range for each numerical attribute

```
range_values = iris_web.max(numeric_only=True) - iris_web.min(numeric_only=True)
print(range_values)
```

9. Interquartile Range (IQR)

```
# First and third quartiles
Q1 = iris_web.quantile(0.25, numeric_only=True)
Q3 = iris_web.quantile(0.75, numeric_only=True)

# IQR
IQR = Q3 - Q1
print(IQR)
```

10. Histogram

```
iris_web.hist(figsize=(8,6))
plt.show()
```

11. Frequency Polygon

```
counts, bin_edges = np.histogram(iris_web['sepal_length'], bins=10)
bin_midpoints = (bin_edges[:-1] + bin_edges[1:]) / 2

plt.plot(bin_midpoints, counts, marker='o')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Frequency')
plt.title('Frequency Polygon of Sepal Length')
plt.show()
```

12. Normal Distribution

```
import numpy as np
import matplotlib.pyplot as plt

data = iris_web['sepal_length']
```

```
mu, sigma = data.mean(), data.std()
x = np.linspace(min(data), max(data), 100)

plt.plot(x, (1/(sigma*np.sqrt(2*np.pi))) * np.exp(-0.5*((x-mu)/sigma)**2))
plt.xlabel('Sepal Length')
plt.ylabel('Probability Density')
plt.title('Normal Distribution of Sepal Length')
plt.show()
```

Output :

1.

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

2.

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

3. Mean:

Id 75.500000

SepalLengthCm 5.843333

```
SepalWidthCm      3.054000
PetalLengthCm     3.758667
PetalWidthCm      1.198667
dtype: float64
Median:
 Id              75.50
SepalLengthCm     5.80
SepalWidthCm      3.00
PetalLengthCm     4.35
PetalWidthCm      1.30
dtype: float64

Mode:
      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  \
0      1            5.0           3.0            1.5           0.2
1      2            NaN           NaN            NaN           NaN
2      3            NaN           NaN            NaN           NaN
3      4            NaN           NaN            NaN           NaN
4      5            NaN           NaN            NaN           NaN
..   ...            ...           ..             ...           ...
145  146            NaN           NaN            NaN           NaN
146  147            NaN           NaN            NaN           NaN
147  148            NaN           NaN            NaN           NaN
148  149            NaN           NaN            NaN           NaN
149  150            NaN           NaN            NaN           NaN

            Species
0       Iris-setosa
1    Iris-versicolor
2     Iris-virginica
3               NaN
4               NaN
..              ...
145             NaN
146             NaN
147             NaN
148             NaN
149             NaN
```



```
[150 rows x 6 columns]
```

4. Variance:

Id 1887.500000
SepalLengthCm 0.685694
SepalWidthCm 0.188004
PetalLengthCm 3.113179
PetalWidthCm 0.582414
dtype: float64

Standard Deviation:

Id 43.445368
SepalLengthCm 0.828066
SepalWidthCm 0.433594
PetalLengthCm 1.764420
PetalWidthCm 0.763161
dtype: float64

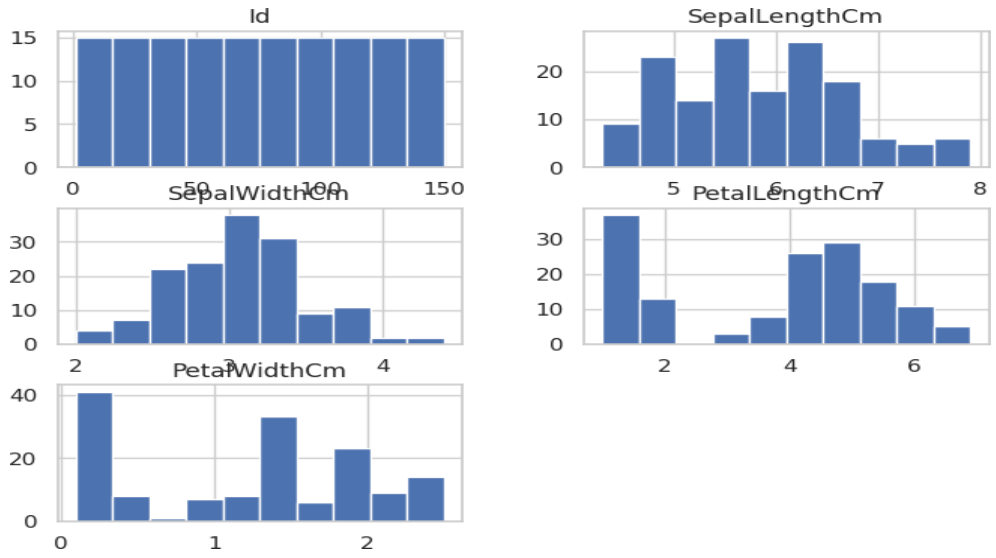
5.

Species	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
Iris-setosa	25.5	5.006	3.418	1.464	0.244
Iris-versicolor	75.5	5.936	2.770	4.260	1.326
Iris-virginica	125.5	6.588	2.974	5.552	2.026

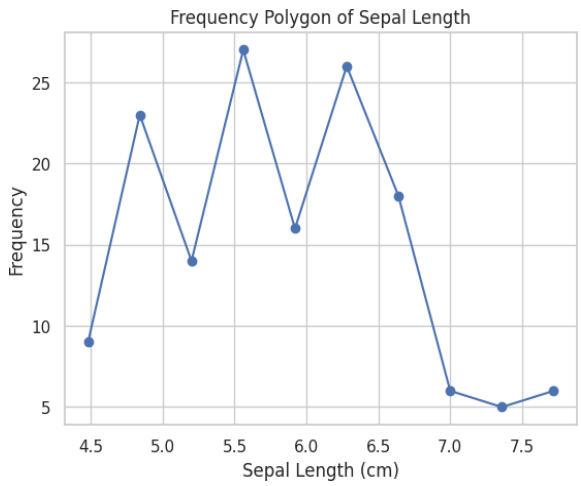
6.Range:

Id 149.0
SepalLengthCm 3.6
SepalWidthCm 2.4
PetalLengthCm 5.9
PetalWidthCm 2.4
dtype: float64

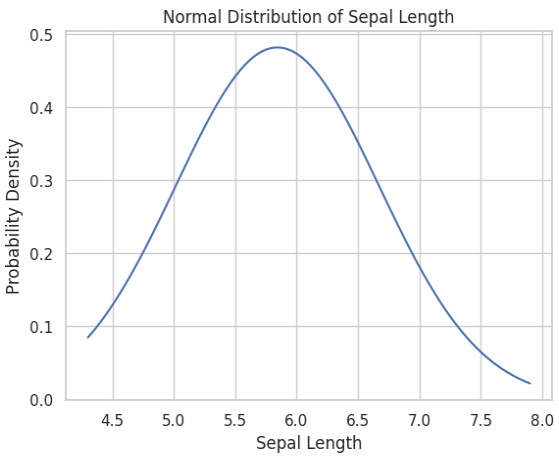
7.


Id
SepalLengthCm
SepalWidthCm
PetalLengthCm
PetalWidthCm

8.


Frequency Polygon of Sepal Length
Frequency
Sepal Length (cm)

9.



CONCLUSION:

Thus, the Iris data set was successfully read from different sources and analyzed using Python to understand its basic statistical properties.

EX NO:4 **DATE:**

UNIVARIATE AND BIVARIATE ANALYSIS ON UCI AND PIMA DATA SET

AIM

To perform Univariate and Biavriate analysis (including frequency, mean, median, mode, variance,standard deviation, skewness and kurtosis on the UCI Indians Diabetes data set.

CODE

```
from google.colab import files
uploaded = files.upload()
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")

data = pd.read_csv("pima.csv.xls")
data.head()
**UNIVARIATE ANALYSIS**
print(data.describe())
print("Mean:", data['Glucose'].mean())
print("Median:", data['Glucose'].median())
print("Std Dev:", data['Glucose'].std())
print(data['Outcome'].value_counts())
data['Glucose'].hist()
plt.title("Glucose Distribution")
plt.show()
sns.boxplot(x=data['BMI'])
plt.title("BMI Boxplot")
plt.show()
data['Outcome'].value_counts().plot(kind='bar')
```

```
plt.title("Outcome Frequency")
plt.show()
```

BIVARIATE ANALYSIS

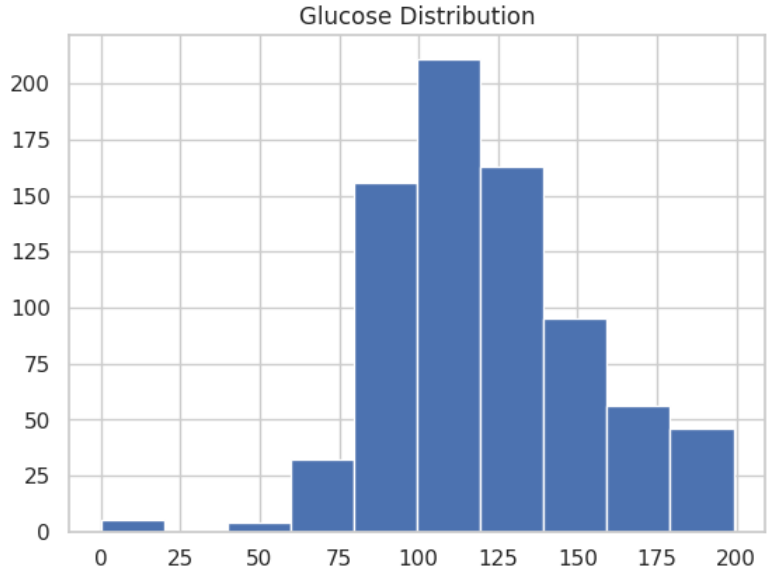
```
sns.scatterplot(x='Glucose', y='BMI', hue='Outcome', data=data)
plt.title("Glucose vs BMI")
plt.show()
corr = data.corr()
print(corr)
sns.heatmap(corr, annot=True)
plt.title("Correlation Heatmap")
plt.show()
sns.pairplot(data[['Glucose','BMI','Age','Outcome']], hue='Outcome')
plt.show()
```

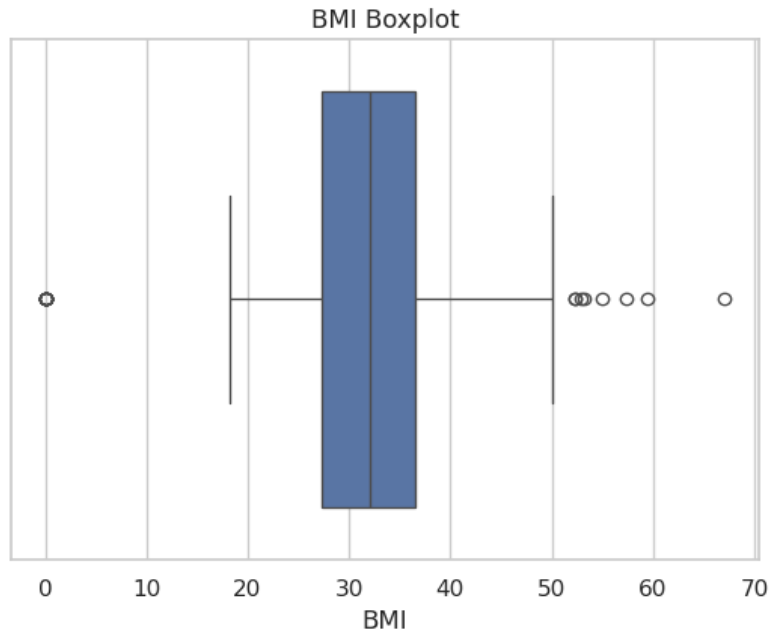
OUTPUT

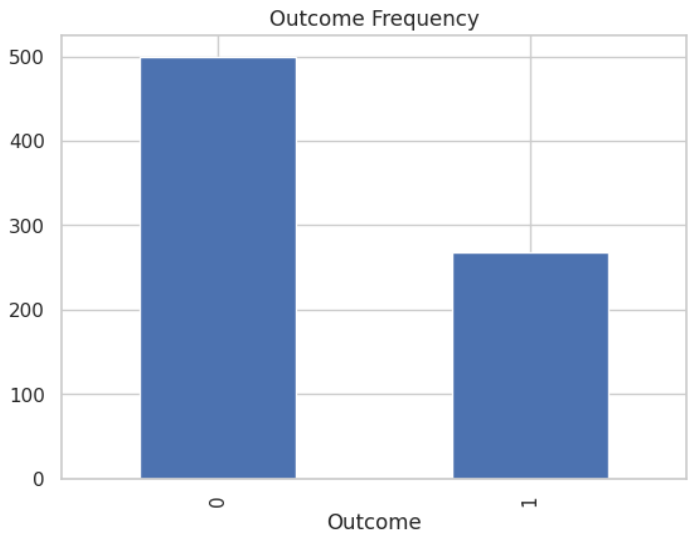
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

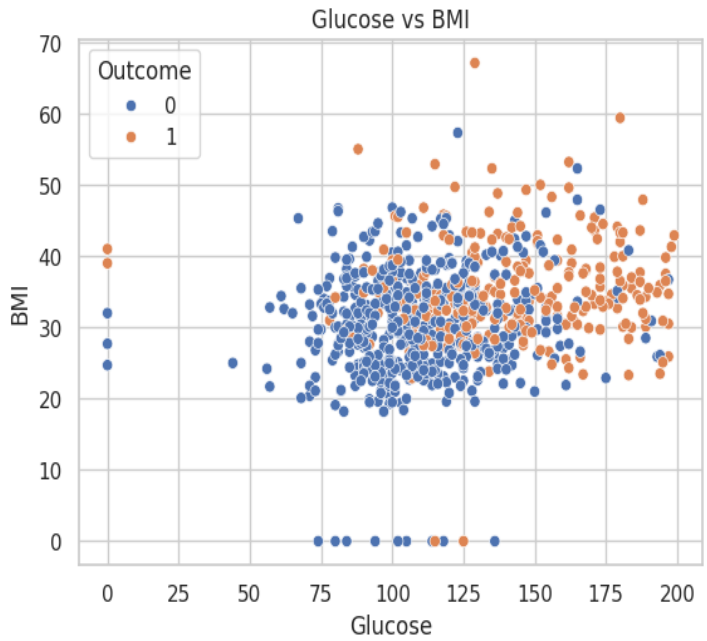
```
Mean: 120.89453125
Median: 117.0
Std Dev: 31.97261819513622
```

```
Outcome
0    500
1    268
Name: count, dtype: int64
```


Glucose Distribution
200
175
150
125
100
75
50
25
0
0
25
50
75
100
125
150
175
200

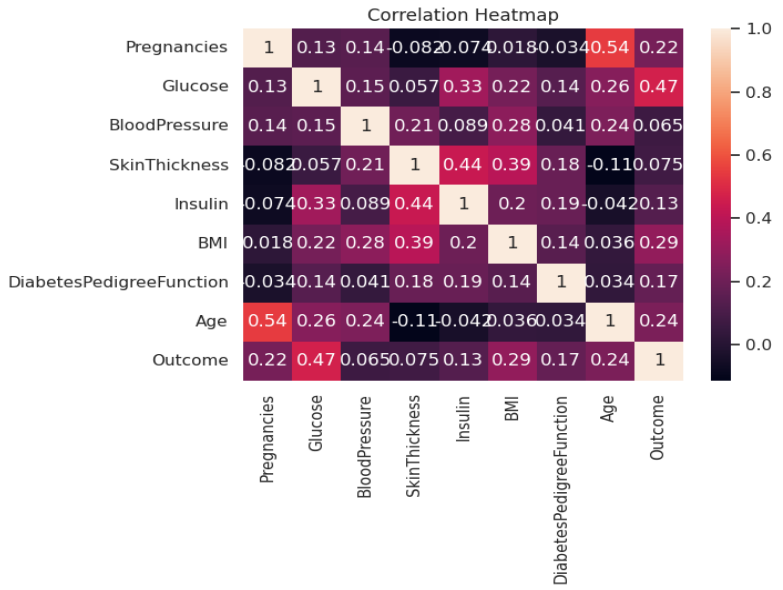

BMI Boxplot
0
10
20
30
40
50
60
70
BMI

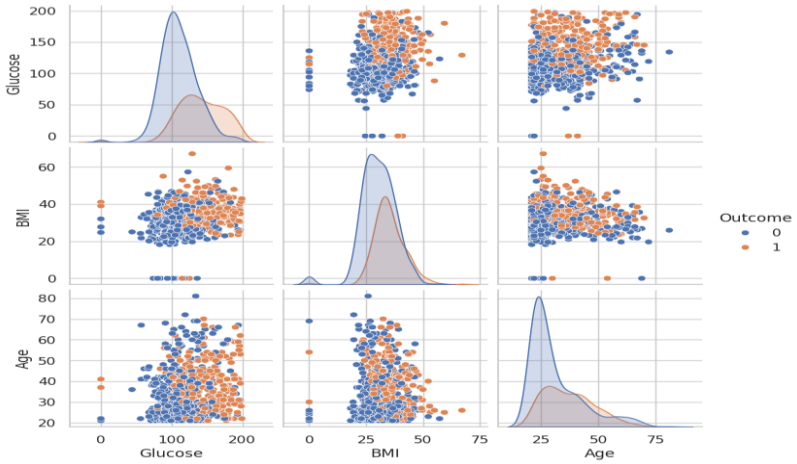





```
                          Pregnancies   Glucose  BloodPressure  SkinThickness  \
Pregnancies                  1.000000  0.129459       0.141282      -0.081672
Glucose                      0.129459  1.000000       0.152590       0.057328
BloodPressure                0.141282  0.152590       1.000000       0.207371
SkinThickness               -0.081672  0.057328       0.207371       1.000000
Insulin                     -0.073535  0.331357       0.088933       0.436783
BMI                          0.017683  0.221071       0.281805       0.392573
DiabetesPedigreeFunction    -0.033523  0.137337       0.041265       0.183928
Age                          0.544341  0.263514       0.239528      -0.113970
Outcome                      0.221898  0.466581       0.065068       0.074752
```

```
                          Insulin       BMI  DiabetesPedigreeFunction  \
Pregnancies             -0.073535  0.017683                 -0.033523
Glucose                  0.331357  0.221071                  0.137337
BloodPressure            0.088933  0.281805                  0.041265
SkinThickness            0.436783  0.392573                  0.183928
Insulin                  1.000000  0.197859                  0.185071
BMI                      0.197859  1.000000                  0.140647
DiabetesPedigreeFunction 0.185071  0.140647                  1.000000
Age                     -0.042163  0.036242                  0.033561
Outcome                  0.130548  0.292695                  0.173844

                              Age   Outcome
Pregnancies              0.544341  0.221898
Glucose                  0.263514  0.466581
BloodPressure            0.239528  0.065068
SkinThickness           -0.113970  0.074752
Insulin                 -0.042163  0.130548
BMI                      0.036242  0.292695
DiabetesPedigreeFunction 0.033561  0.173844
Age                      1.000000  0.238356
Outcome                  0.238356  1.000000
```





CONCLUSION

Thus Univariate and Bivariate analysis was performed on the UCI Indians Diabetes data set, and the results were verified.

EX NO:5a **DATE:**

MULTIPLE REGRESSION ANALYSIS ON PIMA AND UCI DATASET

AIM

To perform Multiple Regression analysis on the diabetes dataset from UCI and Pima Indians Diabetes dataset

ALGORITHM

1. Data Preparation

Input:

Two diabetes datasets (UCI and Pima) in CSV format

Each dataset contains:

Features (multiple medical measurements)

Target variable (diabetes outcome)

Preprocessing:

For UCI dataset: Remove timestamp/datetime column

For both datasets:

Separate features (all columns except last) → X

Separate target (last column) → Y

2. Train-Test Split

Split both X and Y into:

Training set (80% of data)

Test set (20% of data).Use fixed random seed (random_state=42) for reproducibility

3. Feature Standardization

Calculate mean and standard deviation only from training data.Ensures all features have zero mean and unit variance

4. Model Training

Initialize Linear Regression model.Train the model using standardized training dat

5. Model Evaluation

2.

```
     Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0              6      148             72             35        0  33.6
1              1       85             66             29        0  26.6
2              8      183             64              0        0  23.3
3              1       89             66             23       94  28.1
4              0      137             40             35      168  43.1
..           ...      ...            ...            ...      ...   ...
763           10      101             76             48      180  32.9
764            2      122             70             27        0  36.8
765            5      121             72             23      112  26.2
766            1      126             60              0        0  30.1
767            1       93             70             31        0  30.4

     DiabetesPedigreeFunction  Age  Outcome
0                       0.627   50        1
1                       0.351   31        0
2                       0.672   32        1
3                       0.167   21        0
4                       2.288   33        1
..                        ...  ...      ...
763                     0.171   63        0
764                     0.340   27        0
765                     0.245   30        0
766                     0.349   47        1
767                     0.315   23        0

[768 rows x 9 columns]
     Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0              6      148             72             35        0  33.6
1              1       85             66             29        0  26.6
2              8      183             64              0        0  23.3
3              1       89             66             23       94  28.1
4              0      137             40             35      168  43.1
..           ...      ...            ...            ...      ...   ...
763           10      101             76             48      180  32.9
764            2      122             70             27        0  36.8
765            5      121             72             23      112  26.2
766            1      126             60              0        0  30.1
767            1       93             70             31        0  30.4
```

PROGRAM

1. Multiple Regression modelling on UCI Diabetes data set from UCI

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
ds=pd.read_csv("/content/diabetes-uci.csv")
print(ds)
dsnew=ds.drop(['datetime'],axis=1)
X=dsnew.iloc[:,:-1]
print(X)
Y=dsnew.iloc[:,-1]
print(Y)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,Y,test_size=0.2,random_state=42)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)
X_train_std = scaler.transform(X_train)
X_test_std = scaler.transform(X_test)
print('Standardized features for X \n')
print((X_train_std[:2]))
from sklearn.linear_model import LinearRegression
lr=LinearRegression()
lr.fit(X_train_std,y_train)
print('Training Accuracy = ',(lr.score(X_train_std, y_train)))
print('Testing Accuracy = ',(lr.score(X_test_std, y_test)))
```

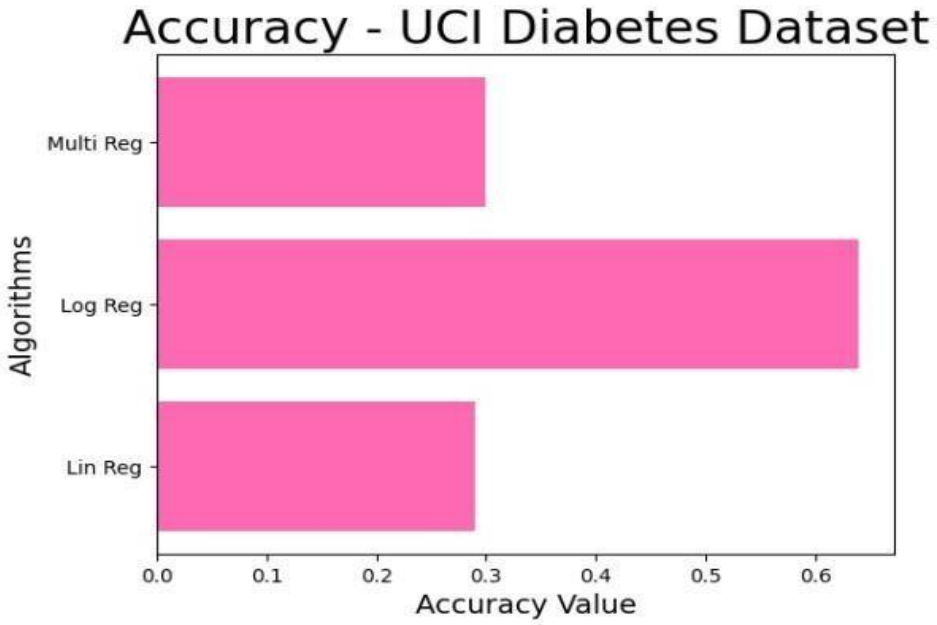
2. Multiple Regression modelling on Pima Indians Diabetes data set

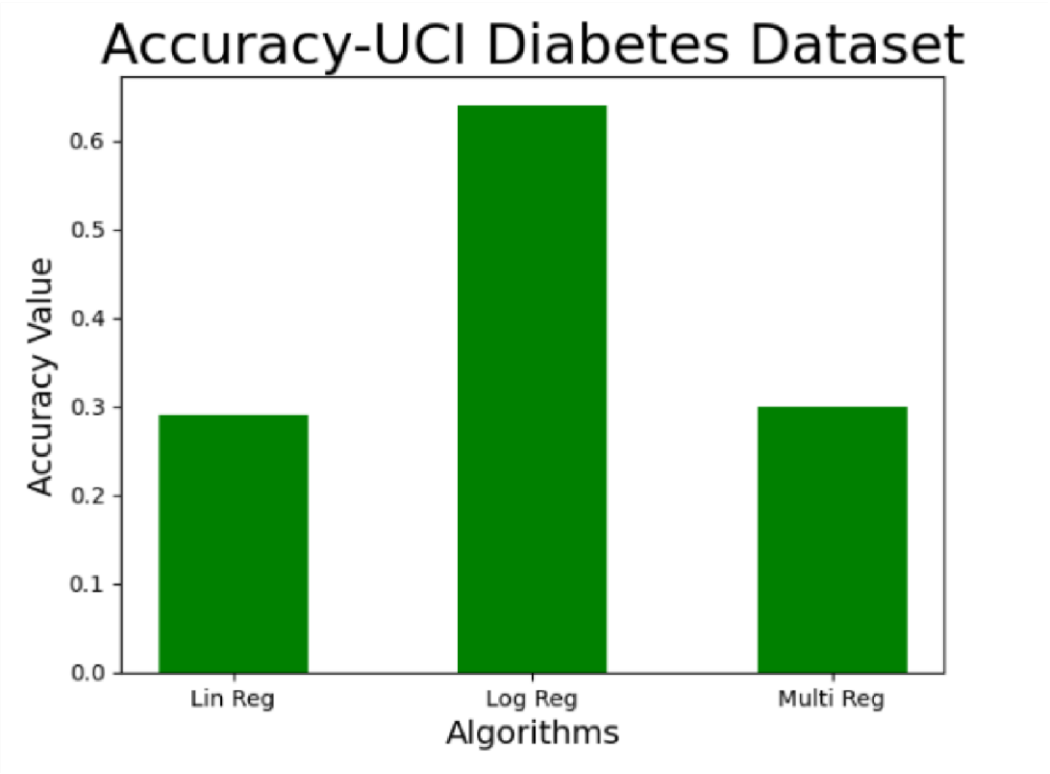
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
     DiabetesPedigreeFunction  Age
0                       0.627   50
1                       0.351   31
2                       0.672   32
3                       0.167   21
4                       2.288   33
..                        ...  ...
763                     0.171   63
764                     0.340   27
765                     0.245   30
766                     0.349   47
767                     0.315   23

[768 rows x 8 columns]
0      1
1      0
2      1
3      0
4      1
      ..
763    0
764    0
765    0
766    1
767    0
Name: Outcome, Length: 768, dtype: int64
Standardized features for X

[[-0.52639686 -1.15139792 -3.75268255 -1.32277365 -0.70120553 -4.13525578
  -0.49073479 -1.03594038]
 [ 1.58804586 -0.27664283  0.68034485  0.23350519 -0.70120553 -0.48916881
   2.41502991  1.48710085]]
Training Accuracy  =  0.30506972801106247
Testing Accuracy  =  0.25500281176741757
```

```
ds=pd.read_csv("/content/diabetes-pima.csv")
print(ds)
X=ds.iloc[:,:-1]
print(X)
Y=ds.iloc[:,-1]
print(Y)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,Y,test_size=0.2,random_state=42)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)
X_train_std = scaler.transform(X_train)
X_test_std = scaler.transform(X_test)
print('Standardized features for X \n')
print((X_train_std[:2]))
from sklearn.linear_model import LinearRegression
lr=LinearRegression()
lr.fit(X_train_std,y_train)
print('Training Accuracy = ',(lr.score(X_train_std, y_train)))
print('Testing Accuracy = ',(lr.score(X_test_std, y_test)))
```

OUTPUT





CONCLUSION

Thus Multiple Regression analysis on the diabetes dataset from UCI and Pima Indians Diabetes dataset,the results were verified.

EX NO:5b **DATE:**

COMPARING THE RESULTS OF ANALYSIS OF PIMA AND UCI DATASETS

AIM

To Compare the results of analysis of Pima and UCI datasets.

ALGORITHM

1. Bring in NumPy for data handling

2. Import Matplotlib for creating plots

3. First Plot (UCI Dataset - Horizontal Bars).Set algorithm names: ["Lin Reg", "Log Reg", "Multi Reg"].Set accuracy values: [0.29, 0.64, 0.30].Create horizontal bar chart (bars go left-to-right).Use pink color for bars.Add title: "Accuracy - UCI Diabetes Dataset".Label x-axis: "Accuracy Value".Label y-axis: "Algorithms".Display the plot

4. Second Plot (UCI Dataset - Vertical Bars)

Use same algorithm names and values.Create vertical bar chart (bars go up-and-down).Use green color for bars.Make bars slightly thinner (width=0.5).Keep same title.

Swap axis labels:

x-axis: "Algorithms"

y-axis: "Accuracy Value"

Display the plot

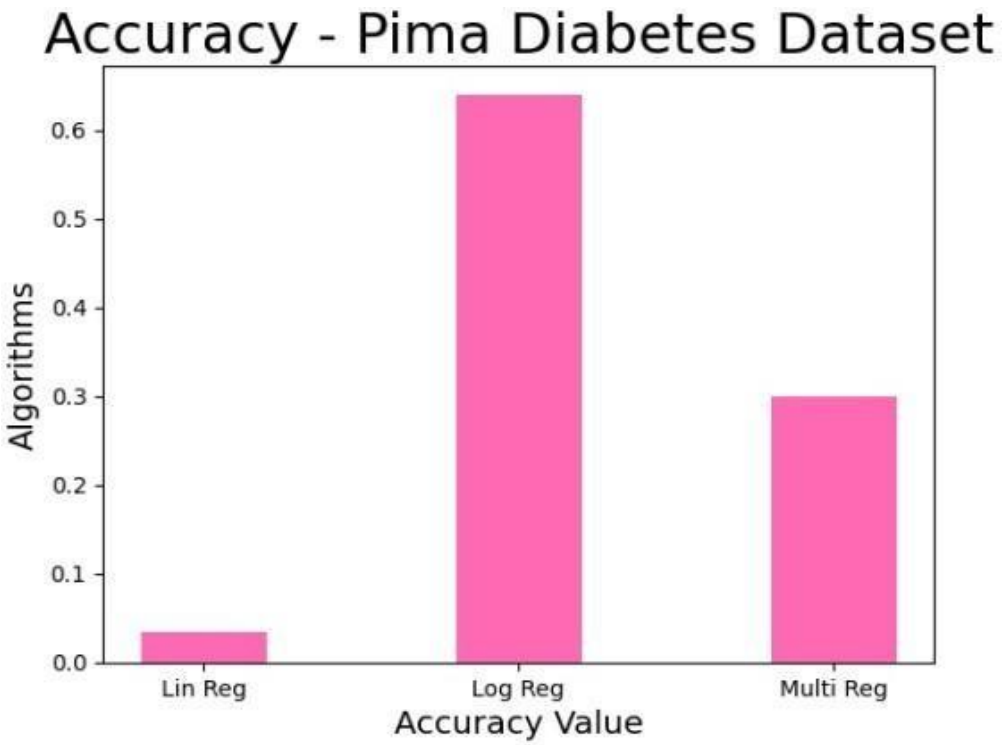
5. Third Plot (Pima Dataset - Vertical Bars).Set new accuracy values: [0.034, 0.64, 0.30].Create vertical bar chart.Use pink color again.Make bars even thinner (width=0.4)

Change title to: "Accuracy - Pima Diabetes Dataset"

Label x-axis: "Accuracy Value"

Label y-axis: "Algorithms"

Display the plot


Accuracy - Pima Diabetes Dataset
Algorithms
0.0
0.1
0.2
0.3
0.4
0.5
0.6
Lin Reg
Log Reg
Multi Reg
Accuracy Value

PROGRAM

Comparison of Accuracy values of three algorithms using Bar plot(Horizontal) - UCI Diabetes Dataset

```
x = np.array(["Lin Reg", "Log Reg","Multi Reg"])
y = np.array([0.29, 0.64,0.30])
plt.title('Accuracy - UCI Diabetes Dataset',fontsize=24)
plt.xlabel('Accuracy Value',fontsize=14)
plt.ylabel('Algorithms',fontsize=14)
plt.barh(x, y,color = "hotpink")
plt.show()
```

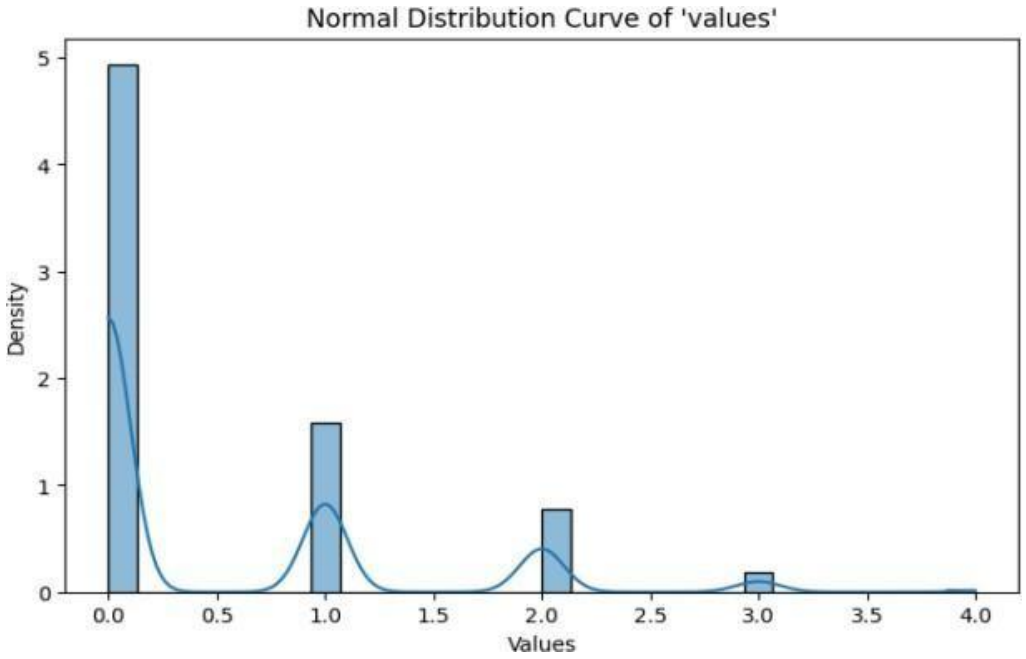
Comparison of Accuracy values of three algorithms using Bar plot(Vertical) - UCI Diabetes Dataset

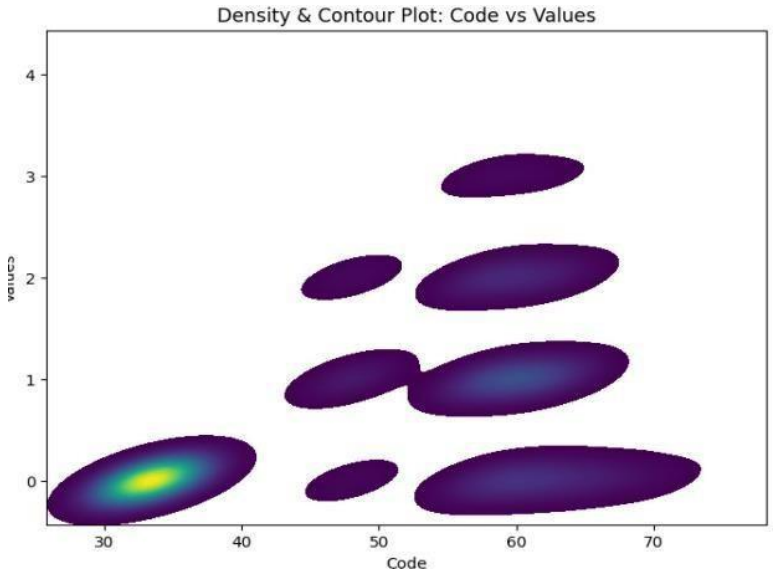
```
x = np.array(["Lin Reg", "Log Reg","Multi Reg"])
y = np.array([0.29, 0.64, 0.30])
plt.title('Accuracy-UCI Diabetes Dataset',fontsize=24)
plt.xlabel('Algorithms',fontsize=14)
plt.ylabel('Accuracy Value',fontsize=14)
plt.bar(x, y,color = "Green", width=0.5)
plt.show()
```

Comparison of Accuracy values of three algorithms using Bar plot- Pima Diabetes Dataset

```
x = np.array(["Lin Reg", "Log Reg","Multi Reg"])
y = np.array([0.034, 0.64, 0.30])
plt.title('Accuracy - Pima Diabetes Dataset',fontsize=24)
plt.xlabel('Accuracy Value',fontsize=14)
plt.ylabel('Algorithms',fontsize=14)
plt.bar(x, y,color = "hotpink",width=0.4)
plt.show()
```

OUTPUT:





CONCLUSION

Thus, comparing results of analysis on the UCI and Pima Indians Diabetes data set were executed and the output was verified.

EX NO:6 **DATE:**

Apply and explore the below plotting functions on UCI data sets

a. Normal curves

b. Density and contour plots

c. Correlation and scatter plots

d. Three–dimensional plotting

Visualizing Geographic Data with Base map

AIM:

To Apply and explore the below plotting functions on UCI data sets Normal curves, Density and contour plots, Correlation and scatter plots, Three–dimensional plotting Visualizing Geographic Data with Base map.

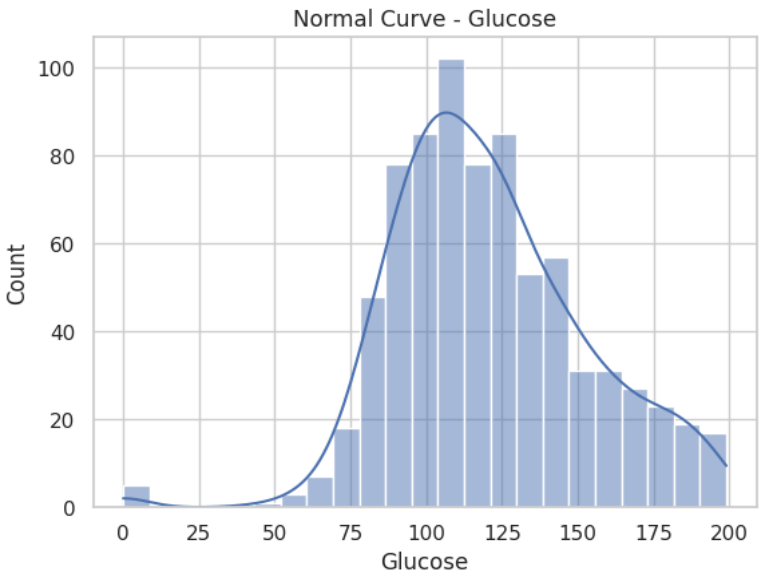
CODE

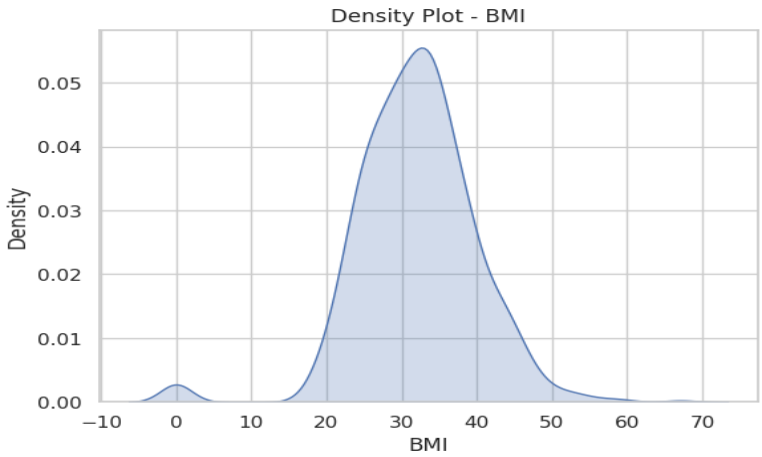
```
from google.colab import files
uploaded = files.upload()
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set(style="whitegrid")
data = pd.read_csv("pima.csv.xls")   # or diabetes.csv
print(data.head())
sns.histplot(data['Glucose'], kde=True)
plt.title("Normal Curve - Glucose")
plt.show()
sns.kdeplot(data['BMI'], fill=True)
plt.title("Density Plot - BMI")
plt.show()
sns.kdeplot(x=data['Glucose'], y=data['BMI'], fill=True)
plt.title("Contour Plot - Glucose vs BMI")
```

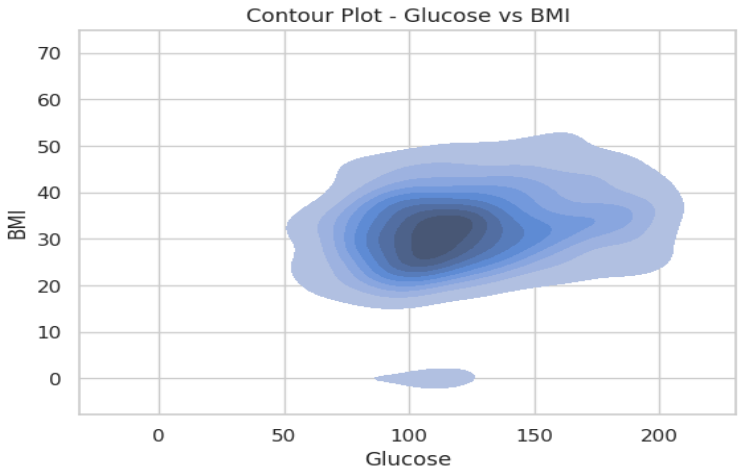
```
plt.show()
corr = data.corr()
print(corr)
sns.heatmap(corr, annot=True)
plt.title("Correlation Heatmap")
plt.show()
sns.scatterplot(x='Glucose', y='BMI', hue='Outcome', data=data)
plt.title("Scatter Plot - Glucose vs BMI")
plt.show()
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(data['Glucose'], data['BMI'], data['Age'])
ax.set_xlabel('Glucose')
ax.set_ylabel('BMI')
ax.set_zlabel('Age')
plt.title("3D Plot - Glucose, BMI, Age")
plt.show()
!pip install basemap
from mpl_toolkits.basemap import Basemap
plt.figure(figsize=(8,6))
m = Basemap(projection='merc',
            llcrnrlat=-60, urcrnrlat=80,
            llcrnrlon=-180, urcrnrlon=180)

m.drawcoastlines()
# Dummy locations (since dataset has no coordinates)
lats = [20, 30, 40]
lons = [70, 80, 90]
x, y = m(lons, lats)
m.scatter(x, y, marker='o')
plt.title("Geographic Visualization using Basemap")
plt.show()
```

OUTPUT

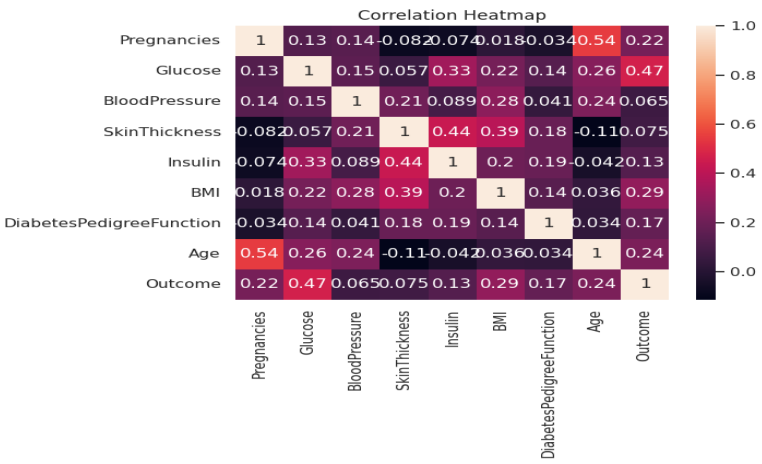
```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

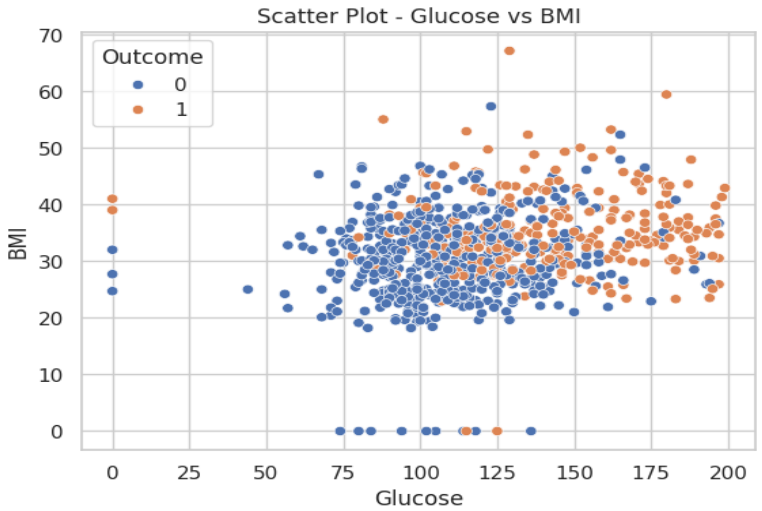


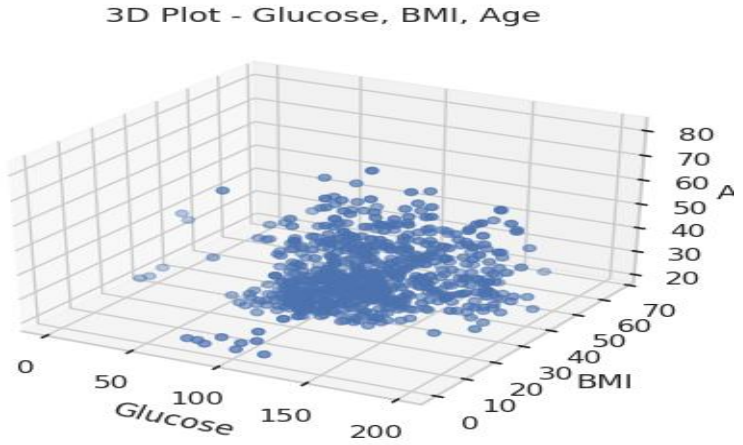


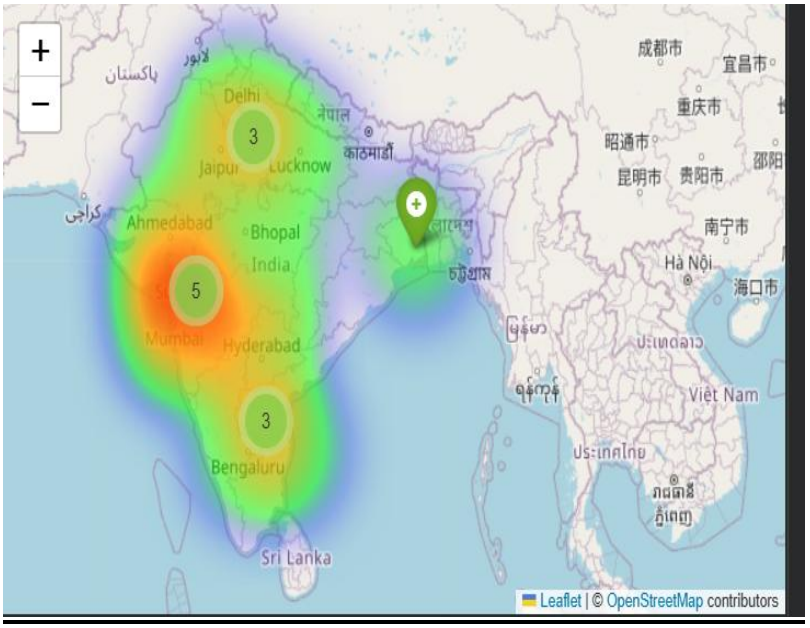



```
                          Pregnancies   Glucose  BloodPressure
SkinThickness  \
Pregnancies                  1.000000  0.129459       0.141282      -
0.081672
Glucose                      0.129459  1.000000       0.152590
0.057328
BloodPressure                0.141282  0.152590       1.000000
0.207371
SkinThickness               -0.081672  0.057328       0.207371
1.000000
Insulin                     -0.073535  0.331357       0.088933
0.436783
BMI                          0.017683  0.221071       0.281805
0.392573
DiabetesPedigreeFunction    -0.033523  0.137337       0.041265
0.183928
Age                          0.544341  0.263514       0.239528      -
0.113970
Outcome                      0.221898  0.466581       0.065068
0.074752

                           Insulin       BMI  DiabetesPedigreeFunction  \
Pregnancies               -0.073535  0.017683                 -0.033523
Glucose                    0.331357  0.221071                  0.137337
BloodPressure              0.088933  0.281805                  0.041265
SkinThickness              0.436783  0.392573                  0.183928
Insulin                    1.000000  0.197859                  0.185071
BMI                        0.197859  1.000000                  0.140647
DiabetesPedigreeFunction   0.185071  0.140647                  1.000000
Age                       -0.042163  0.036242                  0.033561
Outcome                    0.130548  0.292695                  0.173844

                               Age   Outcome
Pregnancies               0.544341  0.221898
Glucose                   0.263514  0.466581
BloodPressure             0.239528  0.065068
SkinThickness            -0.113970  0.074752
Insulin                  -0.042163  0.130548
BMI                       0.036242  0.292695
DiabetesPedigreeFunction  0.033561  0.173844
Age                       1.000000  0.238356
Outcome                   0.238356  1.000000
```


Correlation Heatmap
Pregnancies 1 0.13 0.14 -0.082 0.074 0.018 0.034 0.54 0.22
Glucose 0.13 1 0.15 0.057 0.33 0.22 0.14 0.26 0.47
BloodPressure 0.14 0.15 1 0.21 0.089 0.28 0.041 0.24 0.065
SkinThickness 0.082 0.057 0.21 1 0.44 0.39 0.18 -0.11 0.075
Insulin 0.074 0.33 0.089 0.44 1 0.2 0.19 -0.042 0.13
BMI 0.018 0.22 0.28 0.39 0.2 1 0.14 0.036 0.29
DiabetesPedigreeFunction 0.034 0.14 0.041 0.18 0.19 0.14 1 0.034 0.17
Age 0.54 0.26 0.24 -0.11 -0.042 0.036 0.034 1 0.24
Outcome 0.22 0.47 0.065 0.075 0.13 0.29 0.17 0.24 1
Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome
1.0
0.8
0.6
0.4
0.2
0.0


Scatter Plot - Glucose vs BMI
Outcome
0
1
BMI
Glucose
0
10
20
30
40
50
60
70
0
25
50
75
100
125
150
175
200


3D Plot - Glucose, BMI, Age
Glucose
BMI
A
0
50
100
150
200
0
10
20
30
40
50
60
70
20
30
40
50
60
70
80



CONCLUSION:

Thus, the various plotting functions on UCI data sets were applied and the outputs were verified.